PES University, Bengaluru

(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - May 2023

UE20CS353 - Compiler Design

Total Marks : 100.0

1.a. Answer the following :

1. Briefly state the role of a lexer. Is it possible to have a common lexer for all the languages? Justify your answer with at least 2 examples. **[4 marks]**

2. Provide an example where the lexer requires help from the other phases of the compiler, in order to identify the correct role(or we can say token name) of a currently matched lexeme. **[2 marks]**

3. What is the function of an Error Handling routine? What are the different ways in which error handling can be implemented by a compiler (i.e. the common error recovery methods)? **[4 marks]** (10.0 Marks)

1.b. Answer the following :

1. What is the purpose of Transition diagram in the design of a lexer? **[2 marks]**

2. What is a regular definition? Explain with an example. What is the purpose of using a regular definition? **[3 marks]**

3. Show how a lexical analyzer tool internally simulates the action of an automaton (NFA/DFA) to identify the parts in the input string let's say “**abbca**” that match the following patterns :

```
ab?      printf("1");
ab?b*    printf("2");
bc       printf("3");
```

Highlight the places where you get ambiguity while scanning the input string and specify how it can be resolved. **[5 marks]**

Note : You must trace the automata and provide scanning details. Recall the topic "Design of a Lexical Analyzer Generator" (10.0 Marks)

2.a. Answer the following :

1. With the help of a diagram explain how the different parsers that you have studied in the Compiler Design course compare against each other.
[4 marks]

2. Consider the following grammar:
S → Ae | AB | dB
A → d | f
B → g
where the terminals are {d, e, f, g}.

Is the grammar in LR(0)? In SLR(1)? Justify your answer. Draw the LR(0) automata.
[6 marks] (10.0 Marks)

2.b. Answer the following :

1. Left factor and remove immediate left recursion in the following grammar:
S --> cA | cBe | cBd | Sc | SA
A --> e
B --> f
[5 marks]

2. Is the following grammar in LL(1) ? Construct its LL(1) top-down parser table.
S --> aSA | λ
A --> abS | c
[5 marks] (10.0 Marks)

3.a. Given the following grammar, add semantic actions that computes the number of sequences of successive a’s.
For example, both acab and aacabb have two sequences of a’s each, while abacabcb have three such sequences and caab has 1 such sequence.

S --> T S | λ
T --> a T b | b T c | c T a | λ

You must use the following attributes(their types have been specified for some clarity) only:

int val,
bool left, and
bool right.

Justify that your rules are working correctly by running your SDD over the input string “aacabb” [output annotated parse tree]

Is your SDD S-attributed or L-attributed? (10.0 Marks)

3.b. Answer the following in a line or two:

1. What is an inherited attribute? **[2 marks]**

2. What does L stand for in L-attributed SDD? If the SDD is not L-attributed does that mean it can never be evaluated? **[2 marks]**

3. What is the procedure to convert any SDD to SDT scheme? **[2 marks]**

4. Consider the following grammar along with translation rules. **[4 marks]**

E --> E1 # T {E.val = E1.val * T.val}
E --> T {E.val = T.val}
T --> T1 % F {T.val = T1.val / F.val}
T --> F {T.val = F.val}
F --> id {F.val = id.val}

Here # and % are operators and id is a token that represents an integer and id.val represents the corresponding integer value. The set of non-terminals is {E,T,F} and a subscripted non-terminal indicates an instance of the non-terminal. Using this translation scheme, the computed value of E.val for root of the parse tree for the expression 20 # 10 % 5 # 8 % 2 % 2 is ___________. (10.0 Marks)

4.a. Answer the following :

1. Convert the following to SSA form: **[5 marks]**

```
x = u - t;
y = x * v;
if(y > 100)
        x = y + w;
else
        x = y - w;
y = t - z;
y = x * y;
```

2. What is the disadvantage of using Quadruple format? Express the following three address code statements in Quadruple format : **[5 marks]**

```
param x
param y
t1 = call add, 2
sum = t1
```

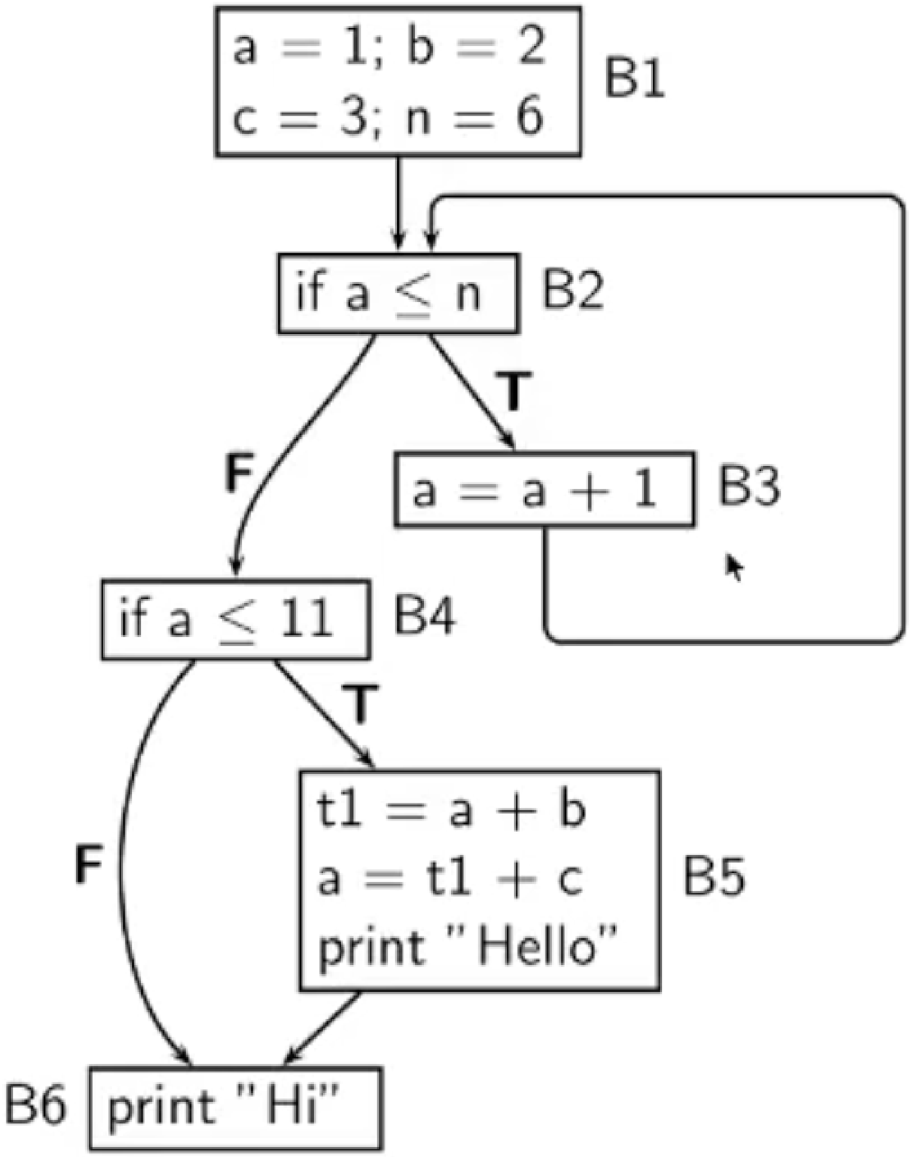
(10.0 Marks)

4.b. Answer the following :

1. Explain the meaning of following optimizations with an example: **[5 marks]**
   1. Constant Folding and Constant Propagation
   2. Strength Reduction
   3. Dead code elimination

2. Perform Live-Variable Analysis on the following CFG: **[5 marks]**

(10.0 Marks)



5.a. Answer the following :

1. What is meant by Calling sequence and Return Sequence? **[5 marks]**

2. Construct an Activation Tree for the following Program: **[5 marks]**

```
int main()
{
 f1();
 f2(2);
 f3();
 return 0;
}

int f1()
{
return 1;
}

int f2(int X)
{
f3();
if(X == 1)
   return f1();
else
    return (X * f2(X - 1));
}

int f3()
{
 return 5;
}
```

(10.0 Marks)

5.b. Answer the following :
1. Explain the tasks which are typically part of a sophisticated compiler's "code generation" phase. **[5 marks]**

2. Using Simple Code Generator algorithm, generate target code sequence for the given statement in a high level language
d = (a - b) + (a - c) + (a - c)

Assume number of registers available are 3 i.e. R1, R2 and R3. All the variables (a, b, c, d) are live on exit from the block.
[5 marks] (10.0 Marks)